Generative Models

Last modified: 2026-10-31

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
source("../tools/use-torch.R")
use_torch(c("numpy", "pandas", "matplotlib"))
```

Brian Hutchinson's CSCI 481/581 sets the order of topics in these notes, which follow it with his permission; this page covers his module on generative models[1]. The exercises are new.

1 Welcome

1.1 Plan for this lecture

The language models chapter drew new text one character at a time from a model of the next character. This lecture draws new images and new data points, with generative models[2] made of neural networks:

1. an autoencoder[3], and why it does not say how to make a new image;
2. a variant whose codes follow a known distribution, so that a new image is a decoded draw from it;
3. two networks trained against each other, one making data and one spotting the fakes;
4. a network that learns to remove noise, and makes data by removing it a step at a time.

1.2 The digits

```
import math

import matplotlib.pyplot as plt
import pandas as pd
import torch
import torch.nn.functional as F
from torch import nn

digits = pd.read_csv("data/optdigits-test.csv", header=None)
x = torch.tensor(digits.iloc[:, :64].to_numpy(), dtype=torch.float32) / 16
y = torch.tensor(digits[64].to_numpy())
_ = torch.manual_seed(481)
order = torch.randperm(len(x))
n_train = 1200
```

[1] https://morrison-lab.github.io/lds/chapters/learning-problems.html#def-generative-model
[2] https://morrison-lab.github.io/lds/chapters/learning-problems.html#def-generative-model
[3] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-autoencoder

```
x_train, y_train = x[order[:n_train]], y[order[:n_train]]
x_test = x[order[n_train:]]
n_test = len(x_test)


def mlp(*sizes):
    # Linear layers of the given sizes, with a ReLU between each pair.
    layers = []
    for n_in, n_out in zip(sizes[:-1], sizes[1:]):
        layers += [nn.Linear(n_in, n_out), nn.ReLU()]
    return nn.Sequential(*layers[:-1])


def train_model(model, loss_fn, n_epochs=100, lr=0.003, seed=0):
    # Adam on minibatches of 64 training images; loss_fn(model, batch) is the loss.
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    generator = torch.Generator().manual_seed(seed)
    for epoch in range(n_epochs):
        shuffled = torch.randperm(n_train, generator=generator)
        for start in range(0, n_train, 64):
            loss = loss_fn(model, x_train[shuffled[start:start + 64]])
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()


def show_digits(images, n_cols=8):
    # Draws each row of `images` as an 8 by 8 grayscale picture.
    n_rows = math.ceil(len(images) / n_cols)
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols, n_rows))
    for ax, image in zip(axes.flat, images):
        ax.imshow(image.reshape(8, 8), cmap="gray_r", vmin=0, vmax=1);
    for ax in axes.flat:
        ax.axis("off");
    plt.show()


print(x_train.shape, x_test.shape)
```

```
torch.Size([1200, 64]) torch.Size([597, 64])
```

The digits are the handwritten digits of the feedforward networks chapter [1], 8 by 8 images with pixel values scaled to lie between 0 and 1. 1,200 images are for training and the other 597 for testing. `mlp()` builds an MLP from a list of layer sizes, and `train_model()` runs Adam[4] on minibatches of 64 training images.

2 Autoencoders

2.1 A two-number code

```
class Autoencoder(nn.Module):
    # Encodes 64 pixels as `code_size` numbers,
```

[4] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-adam

```
    # and decodes those numbers back to 64 pixels between 0 and 1.

    def __init__(self, code_size=2):
        super().__init__()
        self.encoder = mlp(64, 128, code_size)
        self.decoder = nn.Sequential(mlp(code_size, 128, 64), nn.Sigmoid())

    def forward(self, x):
        return self.decoder(self.encoder(x))


def ae_loss(model, batch):
    # Binary cross-entropy of each pixel against its reconstruction,
    # summed over the pixels and averaged over the images.
    return F.binary_cross_entropy(model(batch), batch, reduction="sum") / len(batch)


_ = torch.manual_seed(1)
autoencoder = Autoencoder()
print(autoencoder)
```

```
Autoencoder(
  (encoder): Sequential(
    (0): Linear(in_features=64, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=2, bias=True)
  )
  (decoder): Sequential(
    (0): Sequential(
      (0): Linear(in_features=2, out_features=128, bias=True)
      (1): ReLU()
      (2): Linear(in_features=128, out_features=64, bias=True)
    )
    (1): Sigmoid()
  )
)
```

`Autoencoder` is an autoencoder[5] whose bottleneck has 2 units, so its encoder turns each image into a code of 2 numbers. Other sources call the code a latent representation or latent variable. `ae_loss()` treats each pixel value as the probability that the pixel is dark.

2.2 Reconstruction

Exercise 2.1 (Two numbers per image)**.** Train `autoencoder` with `train_model()` and `ae_loss()`. Is its mean squared error on the test images lower than that of the mean training image, used as the reconstruction of every test image?

Solution

Solution 2.1.

[5] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-autoencoder

```
train_model(autoencoder, ae_loss)
with torch.no_grad():
    ae_mse = F.mse_loss(autoencoder(x_test), x_test).item()
    ae_codes = autoencoder.encoder(x_train)
mean_mse = F.mse_loss(x_train.mean(dim=0).expand_as(x_test), x_test).item()
error_removed = 1 - ae_mse / mean_mse
print(ae_mse, mean_mse, error_removed)
```

```
0.03567703440785408 0.07374946773052216 0.5162401098511429
```

```
# The prose below answers "yes"; stop the render if the autoencoder loses.
assert ae_mse < mean_mse
```

Yes. Its mean squared error per pixel is 0.036, against 0.074 for the mean image. Two numbers per image remove 52% of the mean image's squared error.

2.3 The codes

Exercise 2.2 (Where the codes lie)**.** Do the trained autoencoder[a]'s codes of the training images follow a distribution we could draw new codes from?

[a] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-autoencoder

Solution

Solution 2.2. No.

```
fig, ax = plt.subplots(figsize=(5, 4))
points = ax.scatter(ae_codes[:, 0], ae_codes[:, 1], c=y_train, cmap="tab10", s=6);
fig.colorbar(points, ax=ax, label="digit");
ax.set_xlabel("code 1");
ax.set_ylabel("code 2");
fig.tight_layout();
plt.show()
```

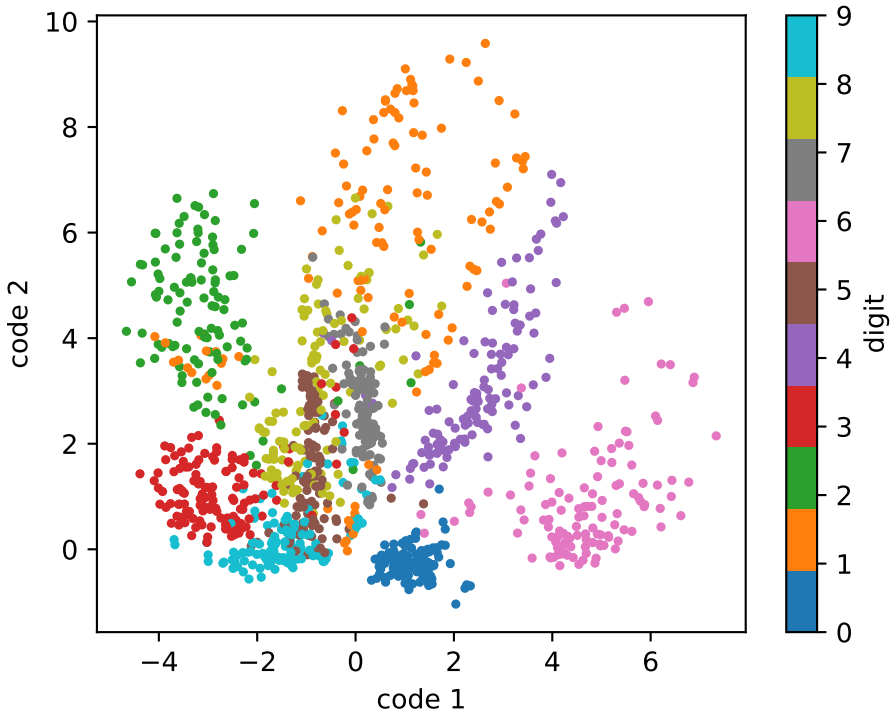


Figure 1: The autoencoder[a]'s 2-number codes of the training images.

```
ae_code_mean = ae_codes.mean(dim=0).tolist()
ae_code_sd = ae_codes.std(dim=0).tolist()
print(ae_code_mean, ae_code_sd)
```

```
[0.015210718847811222, 2.2400617599487305] [2.4187214374542236, 2.1688451766967773]
```

The codes have means 0.02 and 2.24, and standard deviations 2.42 and 2.17. Images of the same digit get nearby codes, and some digits, such as 0, 2 and 6, stand apart, while others, such as 3, 5, 7, 8 and 9, overlap. Nothing in `ae_loss()` says where the codes should lie, so these numbers come out of training, and a different seed would move them. To make a new image we would have to pick a code to decode, and the autoencoder[b] gives no distribution to draw that code from.

[a] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-autoencoder
[b] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-autoencoder

3 Drawing new codes

3.1 A distribution for the codes

Exercise 3.1 (Codes we can draw)**.** The codes of Solution 2.2 follow no distribution we know. What could the encoder output, and the loss add, so that the codes of the training images follow $\mathrm{N}\left(\vec{0}, \mathbb{I}\right)$?

Solution

Solution 3.1. Let the encoder output a normal distribution for each image, with a mean and a standard deviation for each coordinate of the code, and train the decoder on draws from it. Then add to the loss the KL divergence[a] of each of those distributions from $\mathrm{N}(0,1)$. The penalty pulls every image's distribution toward the standard normal, so a draw from $\mathrm{N}\left(\vec{0}, \mathbb{I}\right)$ lands where the decoder has seen codes. This needs two results: the KL divergence between two normal distributions, and a way to differentiate through a random draw.

[a] https://morrison-lab.github.io/pds/expectation.html#def-kl-divergence

3.2 Distance from a standard normal

Exercise 3.2 (How far is a normal distribution from the standard normal?)**.** Let $q = \mathrm{N}(\mu, \sigma^2)$ and $p = \mathrm{N}(0,1)$. Find the KL divergence[a] $D_{\mathrm{KL}}(q \,\|\, p)$ as a formula in μ and σ.

[a] https://morrison-lab.github.io/pds/expectation.html#def-kl-divergence

Solution

Solution 3.2. Let $Z \sim q$, so $\mathbb{E}[Z] = \mu$ and $\mathrm{Var}(Z) = \sigma^2$. The log densities are $\log\{q(z)\} = -\frac{1}{2}\log\{2\pi\} - \frac{1}{2}\log\{\sigma^2\} - \frac{(z-\mu)^2}{2\sigma^2}$ and $\log\{p(z)\} = -\frac{1}{2}\log\{2\pi\} - \frac{z^2}{2}$. Then

$$
\begin{aligned}
D_{\mathrm{KL}}(q \,\|\, p) &= \mathbb{E}[\log\{q(Z)\} - \log\{p(Z)\}] && \text{(definition of KL divergence)} \\
&= \mathbb{E}\left[-\frac{1}{2}\log\{\sigma^2\} - \frac{(Z-\mu)^2}{2\sigma^2} + \frac{Z^2}{2}\right] && \text{(the } \log\{2\pi\} \text{ terms cancel)} \\
&= -\frac{1}{2}\log\{\sigma^2\} - \frac{\mathbb{E}[(Z-\mu)^2]}{2\sigma^2} + \frac{\mathbb{E}[Z^2]}{2} && \text{(linearity of expectation)} \\
&= -\frac{1}{2}\log\{\sigma^2\} - \frac{\sigma^2}{2\sigma^2} + \frac{\mathbb{E}[Z^2]}{2} && (\mathbb{E}[(Z-\mu)^2] = \mathrm{Var}(Z) = \sigma^2) \\
&= -\frac{1}{2}\log\{\sigma^2\} - \frac{\sigma^2}{2\sigma^2} + \frac{\sigma^2 + \mu^2}{2} && (\mathbb{E}[Z^2] = \mathrm{Var}(Z) + \mathbb{E}[Z]^2) \\
&= -\frac{1}{2}\log\{\sigma^2\} - \frac{1}{2} + \frac{\sigma^2 + \mu^2}{2} && \text{(cancel } \sigma^2) \\
&= \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log\{\sigma^2\}). && \text{(collect terms)}
\end{aligned}
$$

Theorem 3.1 (KL divergence from the standard normal)**.** *For $\mu \in \mathbb{R}$ and $\sigma > 0$,*

$$
D_{\mathrm{KL}}(\mathrm{N}(\mu, \sigma^2) \,\|\, \mathrm{N}(0,1)) = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log\{\sigma^2\})
$$

Kingma and Welling [2] derive it in their appendix B.

Proof

Proof. By Solution 3.2. □

Example 3.1 (Checking the formula)**.**

```
from torch.distributions import Normal, kl_divergence

kl_torch = kl_divergence(Normal(1.0, 0.5), Normal(0.0, 1.0)).item()
kl_formula = 0.5 * (1.0**2 + 0.5**2 - 1 - math.log(0.5**2))
print(kl_torch, kl_formula)
```

```
0.8181471824645996 0.8181471805599453
```

For $\mu = 1$ and $\sigma = 0.5$, Theorem 3.1 gives $\frac{1}{2}(1 + 0.25 - 1 - \log\{0.25\}) = 0.8181$, and `torch.distributions.kl_divergence` [3] gives the same value. The divergence is 0 only at $\mu = 0$ and $\sigma = 1$.

3.3 Differentiating through a draw

Exercise 3.3 (A gradient through a random draw)**.** An encoder outputs μ and σ, and the next layer gets a draw z from $\mathrm{N}(\mu, \sigma^2)$. How can we write z so that the gradient[a] of the loss reaches μ and σ?

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

Solution

Solution 3.3. Draw $\varepsilon \sim \mathrm{N}(0, 1)$ first, with no parameters[a] involved, and set $z = \mu + \sigma\varepsilon$. Then $z \sim \mathrm{N}(\mu, \sigma^2)$, and z is a differentiable function of μ and σ, with $\frac{\partial z}{\partial \mu} = 1$ and $\frac{\partial z}{\partial \sigma} = \varepsilon$.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Definition 3.1 (Reparameterization trick)**.** The **reparameterization trick** writes a draw $\underset{\sim}{z}$ from a distribution with parameters[a] $\vec{\theta}$ as $\underset{\sim}{z} = g_{\vec{\theta}}(\underset{\sim}{\varepsilon})$, where $\underset{\sim}{\varepsilon}$ is drawn from a fixed distribution and $g_{\vec{\theta}}$ is differentiable in $\vec{\theta}$ [2, Sec. 2.4]. For a normal distribution with mean $\underset{\sim}{\mu}$ and standard deviations $\underset{\sim}{\sigma}$, $\underset{\sim}{z} = \underset{\sim}{\mu} + \underset{\sim}{\sigma} \odot \underset{\sim}{\varepsilon}$ with $\underset{\sim}{\varepsilon} \sim \mathrm{N}\left(\vec{0}, \mathbb{I}\right)$, where $\odot$ multiplies entry by entry.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

Example 3.2 (Autograd through the draw)**.**

```
mu = torch.tensor(1.0, requires_grad=True)
sigma = torch.tensor(0.5, requires_grad=True)
eps = torch.randn((), generator=torch.Generator().manual_seed(0))
z = mu + sigma * eps
z.backward()
print(eps.item(), mu.grad.item(), sigma.grad.item())
```

```
1.5409960746765137 1.0 1.5409960746765137
```

With the reparameterization trick, autograd finds $\frac{\partial z}{\partial \mu} = 1$ and $\frac{\partial z}{\partial \sigma} = 1.541$, the value of ε drawn.

3.4 The model

Definition 3.2 (Variational autoencoder)**.** A **variational autoencoder** has an encoder that maps an input $\underset{\sim}{x}$ to vectors $\underset{\sim}{\mu}(\underset{\sim}{x})$ and $\underset{\sim}{\sigma}(\underset{\sim}{x})$ in $\mathbb{R}^k$, and a decoder that maps a code $\underset{\sim}{z} \in \mathbb{R}^k$ back to the space of inputs. In training, the code is a draw $\underset{\sim}{z} = \underset{\sim}{\mu}(\underset{\sim}{x}) + \underset{\sim}{\sigma}(\underset{\sim}{x}) \odot \underset{\sim}{\varepsilon}$ by the reparameterization trick. The loss for $\underset{\sim}{x}$ is the reconstruction loss of the decoded $\underset{\sim}{z}$ plus $\sum_{j=1}^{k} D_{\mathrm{KL}}\big(\mathrm{N}(\mu_j(\underset{\sim}{x}), \sigma_j(\underset{\sim}{x})^2) \,\|\, \mathrm{N}(0,1)\big)$ [2, Secs. 2.3, 3]. To make a new input, decode a draw from $\mathrm{N}\big(\vec{0}, \mathbb{I}\big)$.

Example 3.3 (A variational autoencoder for the digits)**.**

```
class VAE(nn.Module):
    # The encoder's 2 * code_size outputs are the means and the log variances.

    def __init__(self, code_size=2):
        super().__init__()
        self.encoder = mlp(64, 128, 2 * code_size)
        self.decoder = nn.Sequential(mlp(code_size, 128, 64), nn.Sigmoid())

    def forward(self, x):
        mu, log_var = self.encoder(x).chunk(2, dim=1)
        z = mu + torch.exp(0.5 * log_var) * torch.randn_like(mu)
        return self.decoder(z), mu, log_var


def vae_loss(model, batch):
    reconstruction, mu, log_var = model(batch)
    fit = F.binary_cross_entropy(reconstruction, batch, reduction="sum")
    kl = 0.5 * (mu**2 + log_var.exp() - 1 - log_var).sum()
    return (fit + kl) / len(batch)


_ = torch.manual_seed(1)
vae = VAE()
train_model(vae, vae_loss)
with torch.no_grad():
    train_mu, train_log_var = vae.encoder(x_train).chunk(2, dim=1)
    noise = torch.randn(train_mu.shape, generator=torch.Generator().manual_seed(2))
    vae_codes = train_mu + torch.exp(0.5 * train_log_var) * noise
    test_mu = vae.encoder(x_test).chunk(2, dim=1)[0]
    vae_mse = F.mse_loss(vae.decoder(test_mu), x_test).item()
vae_code_mean = vae_codes.mean(dim=0).tolist()
vae_code_sd = vae_codes.std(dim=0).tolist()
print(vae_code_mean, vae_code_sd, vae_mse)
```

```
[0.025694118812680244, 0.04258835315704346] [1.0070878267288208, 1.0168691873550415] 0.037325028330087
```

```
# Stop the render if the codes are far from standard normal, as the prose says they are close.
assert all(abs(m) < 0.2 for m in vae_code_mean)
assert all(abs(s - 1) < 0.2 for s in vae_code_sd)
```

`VAE` is a variational autoencoder with codes of 2 numbers, the same layers as `Autoencoder`, and the KL term of Theorem 3.1 added to `ae_loss()`. Its encoder outputs $\log\{\sigma^2\}$ rather than σ, so that any real output gives a valid σ. After the same training, one draw of the code for each training image gives codes with means 0.03 and 0.04 and standard deviations 1.01 and 1.02, close to the means of 0 and

standard deviations of 1 of the standard normal that new codes are drawn from. Decoding $\underset{\sim}{\mu}(\underset{\sim}{x})$ gives a test mean squared error of 0.037, against 0.036 for the autoencoder[a] of Solution 2.1.

[a] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-autoencoder

Remark (The loss as a bound)

Remark 3.1 (The loss as a bound). When the reconstruction loss is the negative log probability that the decoder gives $\underset{\sim}{x}$, as the binary cross-entropy[a] of `vae_loss()` would be for pixels that are each 0 or 1, the negative of the variational autoencoder's loss is a one-draw estimate [2, Sec. 2.3] of what Kingma and Welling [2, Sec. 2.2] call the variational lower bound, a lower bound on the log probability of $\underset{\sim}{x}$. Many sources call it the evidence lower bound (ELBO).

[a] https://morrison-lab.github.io/pds/expectation.html#def-cross-entropy

3.5 Sampling

Exercise 3.4 (New digits)**.** Decode 32 draws from $\mathrm{N}\left(\vec{0}, \mathbb{I}\right)$ with the trained `vae`.

Solution

Solution 3.4.

```
with torch.no_grad():
    new_digits = vae.decoder(torch.randn(32, 2, generator=torch.Generator().manual_seed(4)))
show_digits(new_digits)
```

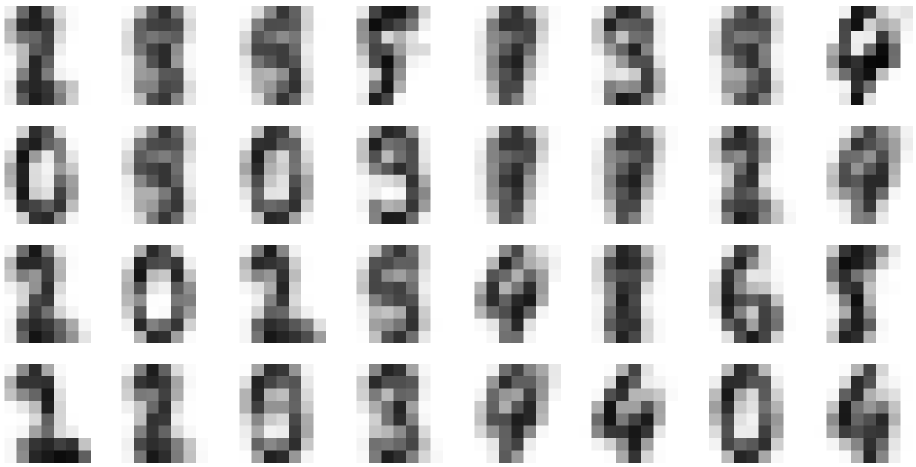


Figure 2: Images decoded from 32 codes drawn from the standard normal distribution.

The variational autoencoder never saw these images. Each is the decoder's output for a code drawn from the distribution that the KL term pulled the training codes toward. They are blurrier than real digits: the decoder's output is a probability for each pixel, and Figure 2 shows those probabilities rather than a draw of dark and light pixels.

4 Two networks against each other

4.1 Old Faithful

```
faithful = pd.read_csv("data/faithful.csv")
eruptions = torch.tensor(faithful.to_numpy(), dtype=torch.float32)
center, scale = eruptions.mean(dim=0), eruptions.std(dim=0)
standardized = (eruptions - center) / scale
long_share = (eruptions[:, 0] > 3).float().mean().item()


def long_wait_sd(points):
    # Standard deviation of the wait, over the eruptions longer than 3 minutes.
    return points[points[:, 0] > 3, 1].std().item()


real_long_sd = long_wait_sd(eruptions)
print(faithful.head())
```

```
   eruptions  waiting
0      3.600       79
1      1.800       54
2      3.333       74
3      2.283       62
4      4.533       85
```

```
print(len(faithful), long_share, real_long_sd)
```

```
272 0.6433823704719543 5.994239330291748
```

```
def plot_eruptions(ax, points, title):
    ax.scatter(points[:, 0], points[:, 1], s=6);
    ax.set_xlabel("eruption (minutes)");
    ax.set_ylabel("wait (minutes)");
    ax.set_title(title);


fig, ax = plt.subplots(figsize=(4, 3.5))
plot_eruptions(ax, eruptions, "Old Faithful");
fig.tight_layout();
plt.show()
```

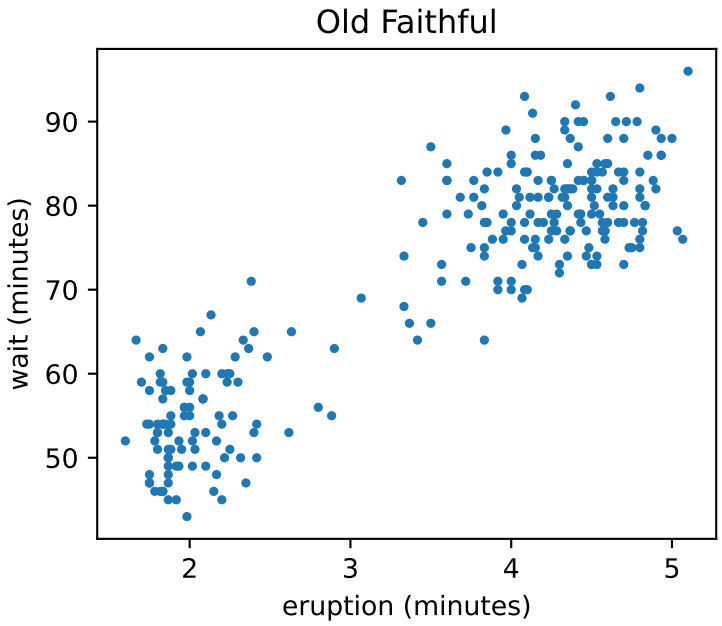


Figure 3: The length of each of the 272 eruptions and the wait until the next one.

The rest of the lecture uses smaller data: a local copy, `data/faithful.csv`[6], of the 272 eruptions of the Old Faithful geyser in R's `faithful` data set [4]. R's documentation gives its source as Härdle [5]; Azzalini and Bowman [6] analyse a more complete version. `tools/get-faithful.R` writes the copy from R, unchanged. Each row gives an eruption's length and the wait until the next one, in minutes. The eruptions fall in two clusters, and 64% of them last longer than 3 minutes. `standardized` rescales each column to mean 0 and standard deviation 1.

4.2 A forger and a detective

> **Exercise 4.1** (Real or made up?)**.** A network turns random noise into made-up eruptions. We have the 272 real eruptions but no formula for their density. How could a second network tell the first one how to make its eruptions look more real?

[6] https://morrison-lab.github.io/dds/chapters/data/faithful.csv

Solution

Solution 4.1. Train the second network as a classifier that tells real eruptions from made-up ones. The first network then changes its weights to make the classifier call its eruptions real. As each network improves, the other has a harder task.

Definition 4.1 (Generative adversarial network (GAN))**.** A **generative adversarial network**, or **GAN**, has a generator G, which maps a draw $\underset{\sim}{z}$ of noise to a data point $G(\underset{\sim}{z})$, and a discriminator D, which gives the probability $D(\underset{\sim}{x})$ that a data point $\underset{\sim}{x}$ is real. They are trained in turn, D to maximize and G to minimize

$$V(D, G) = \mathbb{E}[\log\{D(\underset{\sim}{X})\}] + \mathbb{E}[\log\{1 - D(G(\underset{\sim}{Z}))\}],$$

where $\underset{\sim}{X}$ is a real data point and $\underset{\sim}{Z}$ is the noise [7, Sec. 3].

Example 4.1 (A GAN for Old Faithful)**.**

```
_ = torch.manual_seed(0)
generator_net = mlp(2, 64, 64, 2)
discriminator = mlp(2, 64, 64, 1)
adam = dict(lr=1e-3, betas=(0.5, 0.999))
g_optimizer = torch.optim.Adam(generator_net.parameters(), **adam)
d_optimizer = torch.optim.Adam(discriminator.parameters(), **adam)
draws = torch.Generator().manual_seed(0)
real_label, fake_label = torch.ones(64, 1), torch.zeros(64, 1)
for step in range(3000):
    real = standardized[torch.randint(0, len(standardized), (64,), generator=draws)]
    fake = generator_net(torch.randn(64, 2, generator=draws))
    # The discriminator's step: logits, so the loss is cross-entropy on D.
    d_loss = (
        F.binary_cross_entropy_with_logits(discriminator(real), real_label)
        + F.binary_cross_entropy_with_logits(discriminator(fake.detach()), fake_label)
    )
    d_optimizer.zero_grad()
    d_loss.backward()
    d_optimizer.step()
    # The generator's step: make D call the fakes real.
    g_loss = F.binary_cross_entropy_with_logits(discriminator(fake), real_label)
    g_optimizer.zero_grad()
    g_loss.backward()
    g_optimizer.step()
with torch.no_grad():
    gan_points = generator_net(torch.randn(1000, 2, generator=draws)) * scale + center
gan_long_share = (gan_points[:, 0] > 3).float().mean().item()
gan_long_sd = long_wait_sd(gan_points)
print(gan_long_share, gan_long_sd, real_long_sd)
```

```
0.6340000033378601 3.717714786529541 5.994239330291748
```

```
# Stop the render if the generator's waits spread as much as the real ones,
# since the prose says they spread less.
assert gan_long_sd < real_long_sd
```

```
fig, axes = plt.subplots(1, 2, figsize=(8, 3.5), sharex=True, sharey=True)
plot_eruptions(axes[0], eruptions, "real");
plot_eruptions(axes[1], gan_points, "GAN");
fig.tight_layout();
plt.show()
```

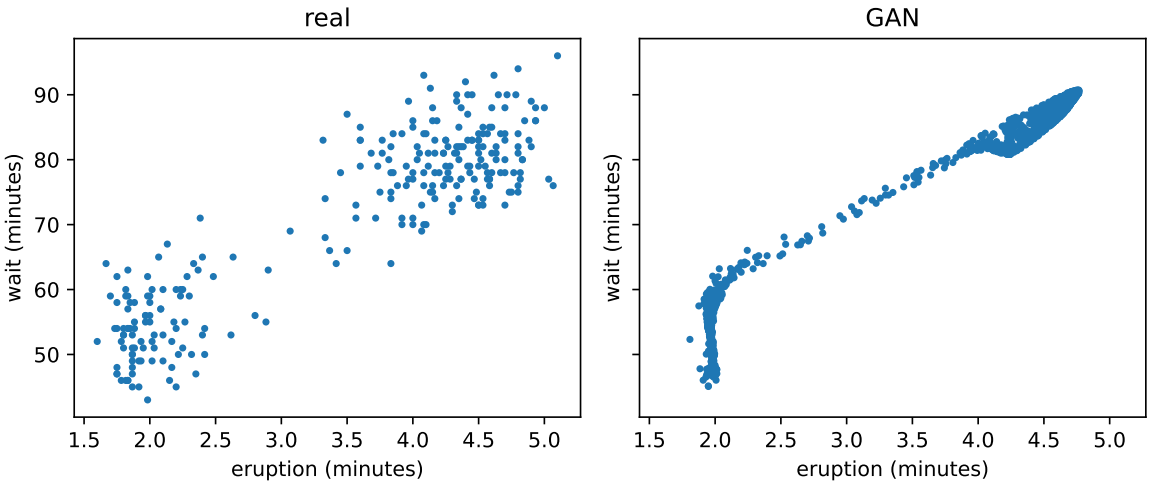


Figure 4: The real eruptions and 1,000 made by the GAN's generator.

Both networks are MLPs with two hidden layers of 64 units, trained for 3,000 steps on the `standardized` eruptions. The discriminator returns a logit, so `binary_cross_entropy_with_logits`, with label 1 for the real points and 0 for the made-up ones, is $-V(D, G)$ for a minibatch. The generator's loss is $-\log\{D(G(\underset{\sim}{z}))\}$, rather than $V(D, G)$. Of the made-up eruptions, 63% last longer than 3 minutes, against 64% of the real ones. But the made-up points lie along one thin curve. Along the long end of that curve, the generator gives each eruption length nearly one wait; along the short end, it gives nearly one length for a range of waits. Among the eruptions longer than 3 minutes, the waits have standard deviation 3.7 minutes, against 6.0 for the real ones. The generator matches where the data lie better than how they spread.

i Remark (The generator's loss in practice)

Remark 4.1 (The generator's loss in practice). Example 4.1 follows Goodfellow et al. [7, Sec. 3], who train G to maximize $\log\{D(G(\underset{\sim}{Z}))\}$ instead of minimizing $V(D, G)$. This gives larger gradients[a] early in training, when D rejects the made-up points easily.

[a] https://morrison-lab.github.io/lds/chapters/linear-regression.html#def-gradient

4.3 The best detective

Exercise 4.2 (The best discriminator)**.** Fix the generator, and let p_g be the density of its made-up points and p that of the real ones. Which $D(\underset{\sim}{x})$ maximizes $V(D, G)$ of Definition 4.1?

Solution

Solution 4.2. A point $G(\underset{\sim}{Z})$ made up by the generator has density p_g, so $\mathbb{E}[\log\{1 - D(G(\underset{\sim}{Z}))\}] = \int p_g(\underset{\sim}{x}) \log\{1 - D(\underset{\sim}{x})\}\, d\underset{\sim}{x}$. Writing the first expectation the same way puts V in one integral over $\underset{\sim}{x}$:

$$V(D,G) = \int \big(p(\underset{\sim}{x})\log\{D(\underset{\sim}{x})\} + p_g(\underset{\sim}{x})\log\{1 - D(\underset{\sim}{x})\}\big)\, d\underset{\sim}{x}.$$

The integral is largest when $D(\underset{\sim}{x})$ maximizes the integrand at each $\underset{\sim}{x}$ separately. With $a = p(\underset{\sim}{x})$, $b = p_g(\underset{\sim}{x})$ and $u = D(\underset{\sim}{x})$, the integrand is $f(u) = a\log\{u\} + b\log\{1-u\}$ on $0 < u < 1$. For $a, b > 0$,

$$\begin{aligned}
f'(u) &= \frac{a}{u} - \frac{b}{1-u} && \text{(differentiate)}\\
0 &= \frac{a}{u} - \frac{b}{1-u} && \text{(set } f'(u) = 0\text{)}\\
0 &= a(1-u) - bu && \text{(multiply by } u(1-u)\text{)}\\
0 &= a - (a+b)u && \text{(collect the terms in } u\text{)}\\
u &= \frac{a}{a+b}. && \text{(solve for } u\text{)}
\end{aligned}$$

$f''(u) = -a/u^2 - b/(1-u)^2 < 0$, so that u is the maximum. Where $b = 0$ the integrand increases toward $u = 1$, and where $a = 0$ it increases toward $u = 0$; $a/(a+b)$ gives those values too. The best discriminator is $D(\underset{\sim}{x}) = p(\underset{\sim}{x})/\big(p(\underset{\sim}{x}) + p_g(\underset{\sim}{x})\big)$. Where the generator's density equals the data's, the best discriminator is $\frac{1}{2}$.

Theorem 4.1 (The optimal discriminator)**.** *For a fixed generator whose points have density p_g, and data with density p, the discriminator that maximizes $V(D,G)$ is*

$$D^*(\underset{\sim}{x}) = \frac{p(\underset{\sim}{x})}{p(\underset{\sim}{x}) + p_g(\underset{\sim}{x})}$$

Goodfellow et al. [7, Sec. 4.1] prove it as their proposition 1.

Proof

Proof. By Solution 4.2. □

Example 4.2 (Where the GAN's discriminator ended up)**.**

```
with torch.no_grad():
    d_real = torch.sigmoid(discriminator(standardized)).mean().item()
    made_up = generator_net(torch.randn(1000, 2, generator=draws))
    d_fake = torch.sigmoid(discriminator(made_up)).mean().item()
print(d_real, d_fake)
```

```
0.5009837746620178 0.5033958554267883
```

If the generator of Example 4.1 matched the data exactly, Theorem 4.1 says the best discriminator would give $\frac{1}{2}$ to every point. The trained discriminator gives the real eruptions an average probability of 0.50 of being real, and the made-up ones 0.50. The generator does not match the data, as Figure 4 shows, so this is not the best discriminator: a small network, trained a step at a time against a moving generator, has stopped telling the two apart on average, though the theorem says a better one could.

5 Adding and removing noise

5.1 Adding noise

Exercise 5.1 (Adding noise twice)**.** Start from a data point $\underset{\sim}{x}_0$, and add noise twice:

$$\underset{\sim}{x}_1 = \sqrt{1-\beta_1}\,\underset{\sim}{x}_0 + \sqrt{\beta_1}\,\underset{\sim}{\varepsilon}_1, \qquad \underset{\sim}{x}_2 = \sqrt{1-\beta_2}\,\underset{\sim}{x}_1 + \sqrt{\beta_2}\,\underset{\sim}{\varepsilon}_2,$$

with $\underset{\sim}{\varepsilon}_1$ and $\underset{\sim}{\varepsilon}_2$ independent draws from $\mathrm{N}\left(\vec{0}, \mathbb{I}\right)$. What is the distribution of $\underset{\sim}{x}_2$ given $\underset{\sim}{x}_0$?

Solution

Solution 5.1. Substitute $\underset{\sim}{x}_1$ into $\underset{\sim}{x}_2$:

$$\begin{aligned}
\underset{\sim}{x}_2 &= \sqrt{1-\beta_2}\left(\sqrt{1-\beta_1}\,\underset{\sim}{x}_0 + \sqrt{\beta_1}\,\underset{\sim}{\varepsilon}_1\right) + \sqrt{\beta_2}\,\underset{\sim}{\varepsilon}_2 && \text{(substitute } \underset{\sim}{x}_1\text{)} \\
&= \sqrt{1-\beta_2}\sqrt{1-\beta_1}\,\underset{\sim}{x}_0 + \sqrt{1-\beta_2}\sqrt{\beta_1}\,\underset{\sim}{\varepsilon}_1 + \sqrt{\beta_2}\,\underset{\sim}{\varepsilon}_2 && \text{(distribute)} \\
&= \sqrt{(1-\beta_1)(1-\beta_2)}\,\underset{\sim}{x}_0 + \sqrt{(1-\beta_2)\beta_1}\,\underset{\sim}{\varepsilon}_1 + \sqrt{\beta_2}\,\underset{\sim}{\varepsilon}_2. && (\sqrt{a}\sqrt{b} = \sqrt{ab})
\end{aligned}$$

The last two terms are independent normal vectors with mean $\vec{0}$, so their sum is normal with mean $\vec{0}$ and covariance

$$\begin{aligned}
((1-\beta_2)\beta_1 + \beta_2)\mathbb{I} &= (\beta_1 - \beta_1\beta_2 + \beta_2)\mathbb{I} && \text{(expand)} \\
&= (1 - (1-\beta_1)(1-\beta_2))\mathbb{I}. && \text{(factor)}
\end{aligned}$$

So $\underset{\sim}{x}_2$ given $\underset{\sim}{x}_0$ is $\mathrm{N}(\sqrt{\bar{\alpha}_2}\,\underset{\sim}{x}_0, (1-\bar{\alpha}_2)\mathbb{I})$, where $\bar{\alpha}_2 = (1-\beta_1)(1-\beta_2)$.

Definition 5.1 (Forward diffusion process)**.** A **forward diffusion process** with noise levels $\beta_1, \ldots, \beta_T$ between 0 and 1 turns a data point $\underset{\sim}{x}_0$ into $\underset{\sim}{x}_1, \ldots, \underset{\sim}{x}_T$ by

$$\underset{\sim}{x}_t = \sqrt{1-\beta_t}\,\underset{\sim}{x}_{t-1} + \sqrt{\beta_t}\,\underset{\sim}{\varepsilon}_t, \qquad t = 1, \ldots, T,$$

with independent $\underset{\sim}{\varepsilon}_t \sim \mathrm{N}\left(\vec{0}, \mathbb{I}\right)$ [8, Sec. 2]. We write $\bar{\alpha}_t \overset{\text{def}}{=} \prod_{s=1}^{t}(1-\beta_s)$; Ho et al. [8, Sec. 2] also write α_t for $1-\beta_t$, which these notes do not use.

Example 5.1 (A noise schedule)**.**

```
n_steps = 100
betas = torch.linspace(1e-4, 0.1, n_steps)
alpha_bars = torch.cumprod(1 - betas, dim=0)
print(alpha_bars[[9, 49, 99]])
```

```
tensor([0.9545, 0.2830, 0.0056])
```

With $T = 100$ steps and β_t rising evenly from 0.0001 to 0.1, $\bar{\alpha}_t$ is 0.95 after 10 steps, 0.283 after 50 and 0.0056 after 100. Ho et al. [8, Sec. 4] use $T = 1000$ steps, with β_t from 0.0001 to 0.02.

Theorem 5.1 (Noise added in one step)**.** *In a forward diffusion process, $\underset{\sim}{x}_t$ given $\underset{\sim}{x}_0$ is $\mathrm{N}(\sqrt{\bar{\alpha}_t}\,\underset{\sim}{x}_0, (1-\bar{\alpha}_t)\mathbb{I})$, so $\underset{\sim}{x}_t = \sqrt{\bar{\alpha}_t}\,\underset{\sim}{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\underset{\sim}{\varepsilon}$ with $\underset{\sim}{\varepsilon} \sim \mathrm{N}\left(\vec{0}, \mathbb{I}\right)$ [8, Sec. 2].*

Proof

Proof. By induction on t. For $t = 1$, $\bar{\alpha}_1 = 1 - \beta_1$ and the statement is the definition. If it holds for $t - 1$, then $\underset{\sim}{x}_t = \sqrt{1-\beta_t}\left(\sqrt{\bar{\alpha}_{t-1}}\,\underset{\sim}{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}}\,\underset{\sim}{\varepsilon}'\right) + \sqrt{\beta_t}\,\underset{\sim}{\varepsilon}_t$, where $\underset{\sim}{\varepsilon}' \sim \mathrm{N}\left(\vec{0}, \mathbb{I}\right)$ depends only on $\underset{\sim}{\varepsilon}_1, \ldots, \underset{\sim}{\varepsilon}_{t-1}$ and so is independent of $\underset{\sim}{\varepsilon}_t$. The steps of Solution 5.1, with $\bar{\alpha}_{t-1}$ in place of $1 - \beta_1$ and β_t in place of β_2, give mean $\sqrt{\bar{\alpha}_t}\,\underset{\sim}{x}_0$ and covariance $(1 - \bar{\alpha}_t)\mathbb{I}$. □

Example 5.2 (Noising the eruptions)**.**

```
fig, axes = plt.subplots(1, 4, figsize=(12, 3), sharex=True, sharey=True)
noise = torch.randn(standardized.shape, generator=torch.Generator().manual_seed(5))
for ax, t in zip(axes, [0, 10, 50, 100]):
    alpha_bar = 1.0 if t == 0 else alpha_bars[t - 1].item()
    noised = math.sqrt(alpha_bar) * standardized + math.sqrt(1 - alpha_bar) * noise
    ax.scatter(noised[:, 0], noised[:, 1], s=4);
    ax.set_title(f"step {t}");
    ax.set_xlabel("eruption (standardized)");
axes[0].set_ylabel("wait (standardized)");
fig.tight_layout();
plt.show()
```

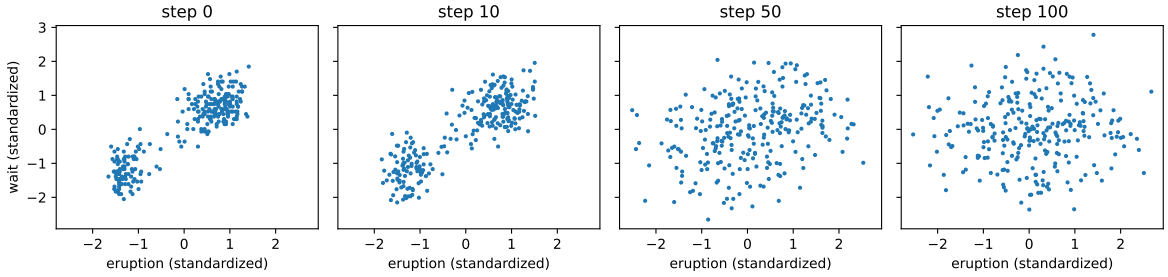


Figure 5: The standardized eruptions after t steps of a forward diffusion process, with the noise schedule of Example 5.1.

Figure 5 draws each step at once from Theorem 5.1, without computing the steps before it. By step 100, $\underset{\sim}{x}_T$ is close to a draw from $\mathrm{N}\left(\vec{0}, \mathbb{I}\right)$.

5.2 Removing noise

Exercise 5.2 (Undoing the noise)**.** A network looks at $\underset{\sim}{x}_t$ and guesses the noise $\hat{\underset{\sim}{\varepsilon}}$ that Theorem 5.1 added to $\underset{\sim}{x}_0$. What estimate of $\underset{\sim}{x}_0$ follows from its guess?

Solution

Solution 5.2. Solve $\underset{\sim}{x}_t = \sqrt{\bar{\alpha}_t}\,\underset{\sim}{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\hat{\underset{\sim}{\varepsilon}}$ for $\underset{\sim}{x}_0$:

$$\hat{\underset{\sim}{x}}_0 = \frac{\underset{\sim}{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\underset{\sim}{\varepsilon}}}{\sqrt{\bar{\alpha}_t}}.$$

A network that predicts the noise well can remove it, so predicting the noise is a way to predict the data.

Definition 5.2 (Denoising diffusion model (denoising diffusion probabilistic model, DDPM))**.** A **denoising diffusion model**, which Ho et al. [8] call a **denoising diffusion probabilistic model** (**DDPM**), is defined for a forward diffusion process. It trains a network $\underset{\sim}{\varepsilon}_{\vec{\theta}}(\underset{\sim}{x}_t, t)$ to predict the noise in $\underset{\sim}{x}_t$, by minimizing $\left\|\underset{\sim}{\varepsilon} - \underset{\sim}{\varepsilon}_{\vec{\theta}}(\sqrt{\bar{\alpha}_t}\,\underset{\sim}{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\underset{\sim}{\varepsilon}, t)\right\|^2$ over data points $\underset{\sim}{x}_0$, steps t drawn uniformly from $1, \ldots, T$, and $\underset{\sim}{\varepsilon} \sim \mathrm{N}\left(\vec{0}, \mathbb{I}\right)$. To make a new point, it draws $\underset{\sim}{x}_T \sim \mathrm{N}\left(\vec{0}, \mathbb{I}\right)$ and steps back, for $t = T, \ldots, 1$:

$$\underset{\sim}{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}}\left(\underset{\sim}{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\underset{\sim}{\varepsilon}_{\vec{\theta}}(\underset{\sim}{x}_t, t)\right) + \sqrt{\beta_t}\,\underset{\sim}{z}_t,$$

with $\underset{\sim}{z}_t \sim \mathrm{N}\left(\vec{0}, \mathbb{I}\right)$ for $t > 1$ and $\underset{\sim}{z}_1 = \vec{0}$ [8, Secs. 3.2, 3.4].

Example 5.3 (A diffusion model for Old Faithful)**.**

```
class NoisePredictor(nn.Module):
    # Predicts the noise in a 2-number point x_t, given t / T as a third input.

    def __init__(self):
        super().__init__()
        self.net = mlp(3, 128, 128, 2)

    def forward(self, x, t):
        return self.net(torch.cat([x, (t / n_steps).unsqueeze(1)], dim=1))


_ = torch.manual_seed(0)
predictor = NoisePredictor()
optimizer = torch.optim.Adam(predictor.parameters(), lr=1e-3)
for step in range(4000):
    x0 = standardized[torch.randint(0, len(standardized), (128,), generator=draws)]
    t = torch.randint(1, n_steps + 1, (128,), generator=draws)
    noise = torch.randn(128, 2, generator=draws)
    alpha_bar = alpha_bars[t - 1].unsqueeze(1)
    noised = alpha_bar.sqrt() * x0 + (1 - alpha_bar).sqrt() * noise
    loss = F.mse_loss(predictor(noised, t.float()), noise)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

with torch.no_grad():
    points = torch.randn(1000, 2, generator=draws)
    for t in range(n_steps, 0, -1):
        beta, alpha_bar = betas[t - 1], alpha_bars[t - 1]
        guess = predictor(points, torch.full((1000,), float(t)))
        points = (points - beta / (1 - alpha_bar).sqrt() * guess) / (1 - beta).sqrt()
        if t > 1:
            points = points + beta.sqrt() * torch.randn(1000, 2, generator=draws)
diffusion_points = points * scale + center
diffusion_long_share = (diffusion_points[:, 0] > 3).float().mean().item()
diffusion_long_sd = long_wait_sd(diffusion_points)
print(diffusion_long_share, diffusion_long_sd)
```

```
0.6230000257492065 5.48890495300293
```

```
fig, axes = plt.subplots(1, 2, figsize=(8, 3.5), sharex=True, sharey=True)
plot_eruptions(axes[0], eruptions, "real");
plot_eruptions(axes[1], diffusion_points, "diffusion");
fig.tight_layout();
plt.show()
```

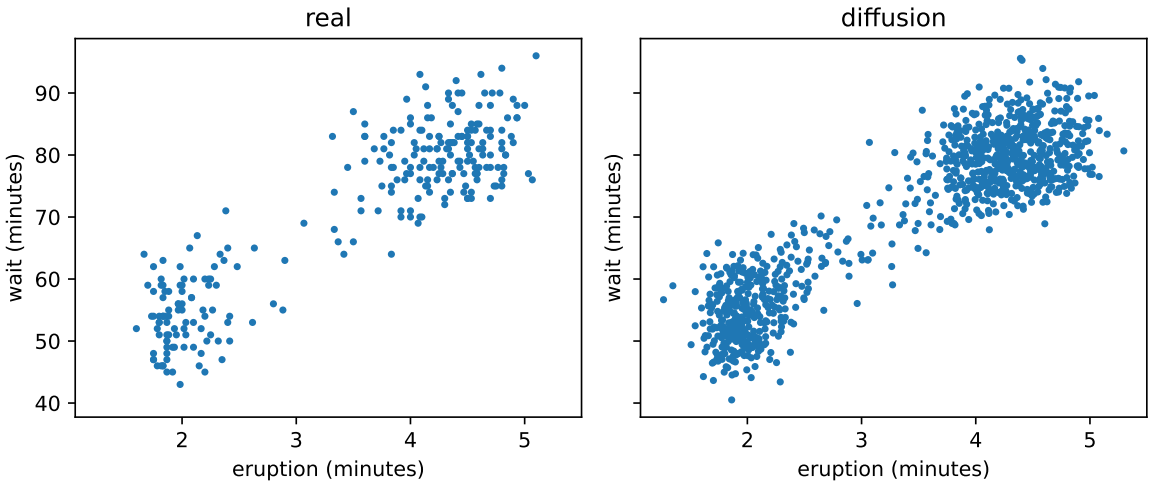


Figure 6: The real eruptions and 1,000 made by the denoising diffusion model.

`NoisePredictor` is an MLP that takes the noised point and t/T, trained for 4,000 steps of the denoising diffusion model's loss on the noise schedule of Example 5.1. Sampling starts from 1,000 draws of pure noise and takes all 100 steps back. Of the points it makes, 62% last longer than 3 minutes, against 64% of the real eruptions. Unlike the GAN's points, they spread around each cluster: among the eruptions longer than 3 minutes, the waits have standard deviation 5.5 minutes, against 6.0 for the real ones.

5.3 Further reading

Read

Deep Learning's chapter 20[a] covers variational autoencoders and GANs [9, Secs. 20.10.3, 20.10.4]. Kingma and Welling [2] introduced the variational autoencoder, Goodfellow et al. [7] the GAN, and Ho et al. [8] the denoising diffusion model.

[a] https://www.deeplearningbook.org/contents/generative_models.html

6 References

[1] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits." UCI Machine Learning Repository, 1998. doi: 10.24432/C50P49[7].

[2] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *2nd international conference on learning representations*, 2014. doi: 10.48550/arXiv.1312.6114[8].

[3] PyTorch Contributors, "Probability distributions: torch.distributions." PyTorch documentation, 2026. Available: https://docs.pytorch.org/docs/stable/distributions.html

[7] https://doi.org/10.24432/C50P49
[8] https://doi.org/10.48550/arXiv.1312.6114

[4] R Core Team, "Faithful: Old Faithful geyser data." R package datasets documentation, 2026. Available: https://stat.ethz.ch/R-manual/R-devel/library/datasets/html/faithful.html

[5] W. Härdle, *Smoothing techniques: With implementation in S*. in Springer series in statistics. New York: Springer, 1991. doi: 10.1007/978-1-4612-4432-5[9].

[6] A. Azzalini and A. W. Bowman, "A look at some data on the Old Faithful geyser," *Applied Statistics*, vol. 39, no. 3, pp. 357–365, 1990, doi: 10.2307/2347385[10].

[7] I. Goodfellow *et al.*, "Generative adversarial nets," in *Advances in neural information processing systems*, 2014. Available: https://papers.nips.cc/paper/5423-generative-adversarial-nets

[8] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in neural information processing systems*, 2020, pp. 6840–6851. Available: https://arxiv.org/abs/2006.11239

[9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/

[9] https://doi.org/10.1007/978-1-4612-4432-5
[10] https://doi.org/10.2307/2347385